

Database Security

1. Why Database Security Is Its Own Discipline

A database often holds an organization's most sensitive, valuable, and regulated data — financial records, personal information, trade secrets. Security here isn't just "keep attackers out" — it's a layered set of concerns: who can even connect, what they're allowed to see, what they're allowed to change, and how to detect and audit misuse after the fact.

The DBA's security responsibilities

- **Account creation & authentication** — who is allowed to connect at all, and verifying they are who they claim to be.
- **Privilege granting/revocation** — deciding what an authenticated user can actually do.
- **Maintaining the audit trail** — a database **log/audit trail** records user activity (who did what, when) — essential both for detecting misuse and for after-the-fact investigation.

Security vs. Precision — a genuine trade-off

Restricting access too **broadly** (denying a user access to an entire table when they only needed to be denied access to *one column*) is **overly imprecise** — it blocks legitimate work unnecessarily. Restricting too **narrowly**/loosely risks leaking sensitive data. Good security design is precise: grant exactly the access actually needed, no more, no less (the **principle of least privilege**, which resurfaces explicitly in the SQL injection defenses below).

2. Discretionary Access Control (DAC)

DAC — access is controlled **at the discretion of data owners**, who can **grant** and **revoke** privileges on data they own to other users, as they see fit.

Two levels of privilege

Level	Applies to	Examples
Account level	The account as a whole, regardless of specific tables	<code>CREATE SCHEMA</code> , <code>CREATE TABLE</code> , <code>CREATE VIEW</code> , <code>ALTER</code> , <code>DROP</code> , <code>MODIFY</code> , <code>SELECT</code>
Relation (table) level	A specific table or view	<code>SELECT</code> , <code>UPDATE</code> / <code>DELETE</code> / <code>INSERT</code> (can be restricted to specific <i>columns</i>), <code>REFERENCES</code>

- The **owner** of a relation (usually its creator) automatically has **all** privileges on it, and can grant them onward.
- Privileges can be modeled as an **access matrix**: rows = users, columns = database objects, each cell = the privileges that user holds on that object.

Using Views to restrict access precisely

If user A wants B to see only **some** rows/columns of table R, A creates a **view** **V** containing exactly that subset, then grants `SELECT` on **V** (not on R itself) to B.

```
CREATE VIEW A3EMPLOYEE AS
  SELECT Name, Bdate, Address FROM EMPLOYEE WHERE Dno = 5;
GRANT SELECT ON A3EMPLOYEE TO A3 WITH GRANT OPTION;
```

B never has any access to the underlying `EMPLOYEE` table directly — only to exactly the filtered slice the view exposes. This is the standard mechanism for precise, row/column-level access control in DAC.

Attribute-specific privileges (a more direct alternative for `UPDATE` / `INSERT` , without needing a view):

```
GRANT UPDATE ON EMPLOYEE (Salary) TO A4;
```

GRANT OPTION — controlled privilege propagation

```
GRANT SELECT ON EMPLOYEE, DEPARTMENT TO A3 WITH GRANT OPTION;
```

- Without `WITH GRANT OPTION` : the recipient can **use** the privilege but **cannot pass it on** to anyone else.
- With it: the recipient can **also grant** that same privilege to others.

Cascading revocation — a crucial consequence: if **A1** revokes `SELECT` from **A3** , and **A3** had previously granted `SELECT` (via its own `GRANT OPTION`) to **A4** , the system must **automatically revoke A4's privilege too** — since it only ever existed because of **A3** 's (now-revoked) privilege.

```
REVOKE SELECT ON EMPLOYEE FROM A3; -- automatically cascades to revoke A4's SELECT too
```

Multi-source privileges: if a user received the *same* privilege from **multiple** independent grantors, it remains valid until **all** of those grantors have revoked it — the DBMS must track the full grant graph to get this right.

Propagation Limits — controlling *how far* a privilege can spread

Type	Controls	Analogy
Horizontal propagation	How many accounts a recipient can grant the privilege to (breadth)	"How many people you can tell a secret to directly"
Vertical propagation	How many levels deep a chain of re-granting can go (depth)	"How many generations the secret can travel through"

Worked example — vertical propagation: A1 grants **SELECT** to A2 with vertical propagation = 2.

- A2 can grant onward to A3, but must set the propagation number to ≤ 1 for that grant.
- A3 can grant further only if its propagation number is still > 0 ; each re-grant must strictly **decrease** the number.
- **Vertical propagation** = 0 is exactly equivalent to **no GRANT OPTION at all** — the chain stops there.

Interview soundbite: "Horizontal and vertical propagation limits answer two genuinely different questions about the same GRANT OPTION mechanism — horizontal asks 'how wide can this spread at any one level?' and vertical asks 'how many hops can this chain travel before it must stop?' You can independently limit either, or both, on the same granted privilege."

3. Mandatory Access Control (MAC) & Multilevel Security

MAC — unlike DAC's all-or-nothing, owner-controlled model, access is governed by **security classifications** assigned to both **users (subjects)** and **data (objects)** — and users **cannot override** these system-enforced rules, no matter what they personally "own."

Classification levels

Top Secret (TS) > Secret (S) > Confidential (C) > Unclassified (U)

Every **user** has a **clearance**; every **object** (a relation, a tuple, even an individual attribute) has a **classification**.

The Bell-LaPadula Model — the two governing rules

1. **Simple Security Property (read rule): "no read up."** A user cannot read data classified **above** their own clearance.
2. **Star (*) Property (write rule): "no write down."** A user cannot **write** data to a classification level **below** their own clearance — this specifically prevents a high-clearance user from *leaking* sensitive information down into a lower-classified object that lower-clearance users can read.

Why "no write down" is just as important as "no read up": without it, a Top-Secret user could simply *copy* Top-Secret information into an Unclassified record, completely bypassing the read restriction for anyone at the Unclassified level — the write rule closes exactly this loophole.

Multilevel Relations & Filtering

A multilevel relation attaches a classification to **each attribute**, plus an overall **tuple classification (TC)** = the highest classification among its attribute classifications:

```
R(A1, C1, A2, C2, ..., An, Cn, TC)
```

Filtering: a user below the classification of a specific attribute sees that attribute's value as **NULL**, rather than the real value or an error — this itself avoids revealing that the field even *has* a hidden non-NULL value versus genuinely being empty.

Polyinstantiation — the trickiest, most important detail

The problem: a filtered (lower-classification) tuple looks, to a low-clearance user, exactly like an ordinary tuple with a NULL field. If that user tries to **update** that field, what should happen?

- The system **cannot** just overwrite the real, higher-classified value — that would literally violate the "no write down... into a value a lower-clearance user controls" spirit, and more importantly:
- The system **cannot simply reject** the update either — doing so would itself leak information! The user could reason: *"my update was rejected... there must be a higher-classification value already sitting here that I'm not allowed to see or touch."* This is a **covert channel** — information about the existence of sensitive data leaking through the system's *behavior*, not through any data value itself.

The fix — polyinstantiation: create a **new, separate tuple** at the lower classification, holding the lower-clearance user's update — **both** tuples now coexist, at different classification levels, sharing the same apparent key.

```
Before: Name=Smith, Job_performance=Excellent, Classification=S

C-level user sees: Name=Smith, Job_performance=NULL
C-level user updates Job_performance = "Good"

After (polyinstantiated):
  Name=Smith, Job_performance=Excellent, Classification=S  ← untouched, still hidden from C-level
  Name=Smith, Job_performance=Good, Classification=C      ← new tuple, visible to C-level
```

The C-level user's update **succeeds** (closing the covert channel — no suspicious rejection), and the S-level data is **never touched or exposed** — both goals achieved simultaneously.

DAC vs. MAC — Summary

	DAC	MAC
Flexibility	High — owners decide at will	Low — strict, system-enforced classification rules
Propagation	Users can pass on access (GRANT OPTION)	No arbitrary propagation — governed entirely by classification
Vulnerability	Susceptible to attacks like Trojan horses (once granted, no control over further misuse)	Very resistant — closes covert channels, prevents any unauthorized flow
Typical use case	General-purpose/commercial databases	Military, government, high-security environments

Interview soundbite: "DAC and MAC aren't mutually exclusive — many real systems layer both: DAC handles the everyday, flexible 'who owns what and who can share it' logic, while MAC enforces an unbreakable, system-wide ceiling that no amount of discretionary granting can ever override. Polyinstantiation is the single cleverest idea in this whole area — it resolves an apparent contradiction (can't reveal, can't reject) by simply admitting that two different 'truths' can coexist at two different classification levels for the same conceptual key."

4. Role-Based Access Control (RBAC)

Instead of granting privileges **directly to individual users**, privileges are assigned to **roles** (e.g., *Sales* , *HR_Manager* , *Reviewer*), and users are then assigned to one or more roles.

Why this matters practically: when an employee's job changes, you simply change their **role assignment** — instead of individually re-auditing and re-granting/revoking a long list of specific privileges accumulated over time (which is exactly the kind of process that goes stale and causes both over-privileged and under-privileged accounts in pure DAC-only systems). RBAC decouples "what

can this job function do" from "who currently holds that job" — a cleaner separation of concerns that scales far better in large organizations.

5. SQL Injection

SQL Injection — an attacker manipulates an application's database queries by injecting malicious SQL through user input fields, tricking the application into executing **unintended** commands.

Methods

Method	How it works	Example
SQL manipulation	Alters an existing query — commonly a <code>WHERE</code> clause, or appending a <code>UNION</code>	Input <code>' OR 'x'='x</code> turns <code>...WHERE password='...'</code> into an always-true condition, bypassing authentication entirely
Code injection	Adds an entirely new SQL statement, exploiting a bug that fails to properly terminate the intended query	Input <code>; DROP TABLE users; --</code> terminates the original query and executes a destructive one
Function call injection	Injects a call to a database or OS-level function, hijacking the flow of execution	In Oracle, injecting a call to a networking function (e.g., <code>UTL_HTTP.REQUEST(...)</code>) can make the <i>database server itself</i> send sensitive data to an attacker-controlled external server

The core vulnerability, in one sentence: SQL injection exploits the **mixing of code and data** — when user input is directly concatenated into a SQL string, the database engine has no way to tell where the *data* ends and the *command* begins.

What attackers gain

- **Database fingerprinting** — determining the specific DBMS in use, to pick known, targeted exploits.
- **Denial of Service** — flooding the server, or deleting critical data.
- **Authentication bypass** — gaining access without ever knowing a valid password.
- **Privilege escalation** — exploiting logical flaws to gain more access than the compromised account should have.
- **Remote command execution** — running arbitrary commands via stored procedures/functions.

Defenses

Technique	How it works	Effectiveness
Parameterized queries / Bind variables	User input is passed as a placeholder value , never concatenated into the SQL string itself — the database treats it strictly as data	The single most important, robust defense — structurally separates code from data at the interpreter level
Input validation/filtering	Sanitize/escape dangerous characters (e.g., single quotes)	Blocks some simple attacks, but unreliable alone — too many possible encodings/escape sequences to filter perfectly
Function security	Restrict which database/OS functions application accounts are even allowed to call	Specifically mitigates function call injection (e.g., prevents even a successfully-injected query from being able to call <code>UTL_HTTP</code>)
Principle of least privilege	The application's own DB account should have the minimum permissions it actually needs (no <code>DROP TABLE</code> rights if it never legitimately drops tables, etc.)	Limits the <i>damage</i> even a successful injection can do
Generic error messages	Never expose detailed database errors to end users	Prevents attackers from using verbose errors to fingerprint the schema/DBMS

Interview soundbite: "Parameterized queries are not just 'one good practice among several' — they fix the root cause (code/data conflation) structurally, at the query-execution layer, so injection becomes essentially impossible regardless of what a user types. Input validation and least-privilege are valuable *defense-in-depth* layers precisely because they limit the blast radius if something else fails — but neither actually prevents the injection itself the way parameterized queries do."

Interview Questions With Answers

Q1. Why is granting access via a view often preferable to granting direct, column-restricted access on the base table?

Answer: A view can express arbitrary combinations of row-filtering and column-selection in a single, reusable object — restricting a user to exactly the rows and columns they should see, defined once. Granting attribute-specific privileges directly on the base table can restrict *which columns* a

user can touch, but doesn't as naturally express row-level restrictions (e.g., "only rows where Dno=5") within the base grant mechanism itself — a view combines both dimensions cleanly, and also insulates the underlying table's actual structure from what's exposed to the grantee.

Q2. Explain why revoking a privilege from a user must sometimes cascade to revoke privileges from other users who never directly received anything from the original grantor.

Answer: If A grants a privilege to B "with GRANT OPTION," B's ability to have granted that same privilege onward to C only ever existed because A's original grant to B included that option. If A later revokes the privilege from B, B no longer legitimately holds it at all — which means B's downstream grant to C was only ever valid because of a privilege B has now lost. To maintain a consistent security state (nobody holding a privilege whose entire chain of authority has been revoked), the system must automatically cascade the revocation down through every privilege that traces its authority back to the now-revoked grant.

Q3. What's the difference between horizontal and vertical propagation limits on a granted privilege?

Answer: Horizontal propagation limits how many *different accounts* a single recipient can grant the privilege to at their own level — controlling the breadth of dissemination at any one step. Vertical propagation limits how many *levels deep* a chain of successive re-grants can go before it must stop — controlling the overall depth of the privilege's lineage. They're independent dimensions: a privilege could be granted with a wide horizontal limit but a shallow vertical limit (spread to many people, but none of them can re-grant further), or vice versa.

Q4. State the two Bell-LaPadula rules, and explain why the "no write down" rule is necessary in addition to "no read up."

Answer: The Simple Security Property ("no read up") says a user cannot read data classified above their own clearance. The Star Property ("no write down") says a user cannot write data to a classification level below their own clearance. "No write down" is necessary because, without it, a high-clearance user (who is fully permitted to *read* sensitive high-classification data) could simply copy that data into a lower-classified object — which lower-clearance users are permitted to read — completely bypassing the read restriction for everyone below. The write rule specifically closes this "high-clearance user leaks data downward" loophole that the read rule alone does nothing to prevent.

Q5. Why can't the system simply reject an update from a lower-clearance user attempting to modify a field that's actually hiding a higher-classification value?

Answer: Rejecting the update, while seemingly the "safe" choice, would itself constitute a covert channel — the lower-clearance user could infer, purely from the fact that their update was refused (when an ordinary NULL field would normally accept an update just fine), that there must be a hidden, higher-classification value already present at that position. This inference leaks the *existence* of sensitive data through the system's behavior, even though no actual data value was ever revealed —

exactly what multilevel security is designed to prevent. This is precisely why polyinstantiation exists: it lets the update succeed (no suspicious rejection) while still never touching or exposing the actual higher-classification data.

Q6. Walk through what polyinstantiation actually does when a C-level user updates a field that secretly holds an S-level value.

Answer: The system does not modify the existing S-level tuple at all — that value remains completely untouched and still hidden from the C-level user. Instead, it creates an entirely new, separate tuple at the C classification level, sharing the same apparent key (e.g., the same Name), containing the C-level user's new value. Both tuples now coexist in the relation simultaneously, each visible only to users at or above their respective classification level — the C-level user sees and can freely interact with "their" tuple, believing their update succeeded normally, while the S-level tuple (and anyone with S clearance viewing it) is entirely unaffected and unaware of the C-level tuple's separate existence.

Q7. Why is RBAC often considered more maintainable than pure DAC for large organizations?

Answer: Under pure DAC, privileges accumulate on individual user accounts over time as people's jobs and needs change, and there's no structural link between "why does this user have this privilege" and "what is their current job function" — auditing and correcting this over time is error-prone, and privileges often go stale (either overly permissive or missing something needed). RBAC decouples the two: privileges are defined once per role (reflecting a job function's actual needs), and a user's access changes automatically and correctly the moment their role assignment changes — there's no need to individually track and re-grant/revoke a long history of accumulated direct privileges per user, which scales far better as an organization and its staff change over time.

Q8. Why do defenses like input validation and least-privilege matter even though parameterized queries are described as the single most important defense against SQL injection?

Answer: Parameterized queries structurally prevent the code/data conflation that SQL injection exploits — but they only protect the specific query paths where they're actually and correctly used; a single overlooked legacy code path built with string concatenation can still be vulnerable. Input validation and least-privilege are "defense in depth" — they don't prevent injection at its root the way parameterization does, but they limit how much *damage* a successful injection (through some other overlooked gap) can actually cause: least privilege ensures even a compromised application account can't do things like drop tables or call dangerous functions it was never granted access to, and input validation can catch some simple attack patterns as an additional layer, even though it's not reliable enough to depend on alone.

Q9. Scenario: A company's internal HR application lets managers update employee records, but the database account the HR application connects with has full `DROP` ,

ALTER, and unrestricted function-execution privileges on the entire database — far more than the application's actual functionality ever needs. An SQL injection vulnerability is later discovered in a search feature of this application. Using the concepts from this topic, explain why the impact of this vulnerability is worse than it needed to be, and what should have been done differently.

Answer: The core injection vulnerability itself (mixing user input directly into SQL, rather than using parameterized queries) is one failure — but the *severity* of what an attacker can actually accomplish through it is separately determined by what privileges the compromised database account holds, which is exactly the principle of least privilege. Because the HR application's database account holds far more privilege than its actual functionality requires (full DROP/ALTER/function-execution rights, when it likely only ever needs SELECT/UPDATE/INSERT on specific employee-related tables), a successful injection through the search feature could be leveraged to drop tables, alter schema, or execute powerful functions — turning what should have been, at worst, unauthorized reads/writes on employee data into a far more catastrophic, database-wide compromise. The fix on two fronts: first, eliminate the injection vulnerability itself via parameterized queries in the search feature; second — independently, as defense in depth — the application's database account should be reconfigured to hold only the minimum privileges its actual features require, so that even if some *other*, still-undiscovered injection vulnerability exists elsewhere in the application, the blast radius of exploiting it is structurally limited to what that account can actually do.